Beyond 99% Accuracy: A Critical Review and Re-evaluation of AI-Based Intrusion Detection for Cloud Environments

Jayveer Vasava M.Tech (Cyber Security), Parul University, Vadodara, Gujarat, India

Abstract

AI-based intrusion detection systems (IDS) for cloud environments are routinely reported to exceed 99% accuracy, yet few of these results are tested under conditions resembling deployment. This paper reviews 30 open-access studies, organised into a taxonomy of classical machine learning, deep learning, hybrid and optimisation methods, feature selection, and explainable and trustworthy AI, and then re-evaluates five representative models (Logistic Regression, Decision Tree, Random Forest, XGBoost and a multilayer perceptron) on the official NSL-KDD and UNSW-NB15 test sets. The best binary F1-scores were 0.798 and 0.897, while a random 80/20 split of the training file alone raised accuracy to 0.998-0.999 and 0.950-0.959, showing that much of the reported performance comes from the evaluation protocol. We then propose RELIC-IDS, five reliability tests and a 10-point reporting checklist. The tests show that unseen (zero-day) attack types were detected at 0.28-0.42 against 0.71-0.79 for known types; false-positive targets fixed on validation data were exceeded 2.2-10 times on the test set; class balancing raised rare-class recall only at the cost of more false alarms; setting the IP time-to-live of attacks to its benign median let 99.8% of detected UNSW-NB15 attacks evade XGBoost; and model size ranged from 1.4 KB to 108 MB. Selecting 30 features by mutual information cut XGBoost training time by 59-74% with an F1 change below 0.01. The paper closes with nine research gaps and matching directions for cloud-native, robust and verifiable intrusion detection.

Index Terms: intrusion detection system, cloud security, machine learning, evaluation methodology, zero-day attacks, adversarial robustness, explainable AI, NSL-KDD, UNSW-NB15.

I. Introduction

Cloud computing delivers storage, compute and software as on-demand services, and the same properties that make it attractive (multi-tenancy, elastic scaling, virtualised networks and remote access) widen its attack surface. A single misconfigured virtual machine (VM), a compromised credential or a distributed denial-of-service (DDoS) flood

can affect many tenants at once. An intrusion detection system (IDS) monitors network traffic or host activity and raises an alert when it observes behaviour that matches an attack or departs from normal operation [1].

Traditional IDS fall into two families. Signature-based systems match traffic against a database of known attack patterns; they are precise for known attacks but blind to zero-day and modified attacks. Anomaly-based systems learn a profile of normal behaviour and flag deviations, which lets them catch new attacks but often at the price of false alarms [1], [2]. In a cloud, both families face additional pressure: traffic volume is high, workloads change constantly, and east-west traffic between VMs may never cross a perimeter sensor [3].

Artificial intelligence (AI), in the form of machine learning (ML) and deep learning (DL), has become the dominant research direction for anomaly-based detection because a model can learn decision boundaries from labelled traffic instead of relying on hand-written rules. Reported accuracies are often above 99%. However, earlier work has warned that ML-based intrusion detection is unusually easy to evaluate optimistically: the closed-world assumption of benchmark datasets, data leakage between training and test sets, and the high cost of each error in an operational setting all make laboratory accuracy a weak predictor of field performance [4], [5].

This paper asks what lies beyond the 99% figure. It combines a structured review with a controlled re-evaluation, so that the claims of the literature can be checked against measured results under a strict protocol, and adds tests of the properties that matter once an IDS is deployed. Its contributions are:

1. A taxonomy and critical review of AI-based IDS for cloud environments built from 30 open-access, verified sources, with a comparison of datasets and of the reported results and protocols.
2. A reproducible re-evaluation of five models on the official NSL-KDD and UNSW-NB15 test sets, quantifying the accuracy inflation caused by random-split evaluation.
3. RELIC-IDS, a set of five reliability tests (zero-day detection, false-alarm control, rare-class balancing, adversarial mimicry and deployment cost) and a 10-point reporting checklist for future cloud IDS studies.
4. Nine research gaps and matching future directions, each tied to evidence from the review or the experiments.

The rest of the paper is organised as follows. Section II gives background on IDS and cloud threats, Section III describes the review method, Section IV presents the taxonomy, Section V the datasets and Section VI the comparative analysis. Section VII reports the baseline re-evaluation and Section VIII the RELIC-IDS tests and checklist. Sections IX to XI discuss gaps, future directions and limitations, and Section XII concludes.

II. Background

A. Types of intrusion detection

IDS are classified by detection method and by where they are deployed [1], [2]:

- **Signature (misuse) based:** matches known attack patterns; low false-alarm rate, cannot detect novel attacks.
- **Anomaly based:** models normal behaviour and flags deviations; can detect novel attacks, prone to false alarms.
- **Hybrid:** combines both, usually a signature stage for known attacks and an anomaly stage for the rest.
- **Network-based (NIDS):** inspects packets or flows at a network point; in a cloud this can be a virtual switch, a VPC flow-log stream or a gateway.
- **Host-based (HIDS):** inspects system calls, logs and files on one host or VM.
- **Hypervisor-based:** observes VMs from the virtualisation layer, which can see inter-VM traffic that never reaches a physical sensor [3].

B. Cloud threat landscape

Cloud IDS research targets DDoS and resource-exhaustion attacks against shared services, probing and scanning of tenant address space, credential abuse and insider misuse, malware and botnet traffic inside tenant networks, and attacks on the virtualisation layer itself [3], [6]. Two cloud properties shape the detection problem. First, scale: a cloud IDS must classify very high flow rates, so training and inference cost matter as much as accuracy. Second, change: workloads are created and destroyed continuously, so the notion of normal traffic drifts and a model trained once degrades over time.

C. A typical AI-based cloud IDS pipeline

Fig. 1 shows the stages most reviewed systems share. Traffic and logs are collected from cloud sources, cleaned and encoded, reduced to an informative feature subset, and classified by an AI model. Recent work adds an explanation stage, so an analyst can see why an alert fired, and a feedback loop that retrains the model as traffic changes.

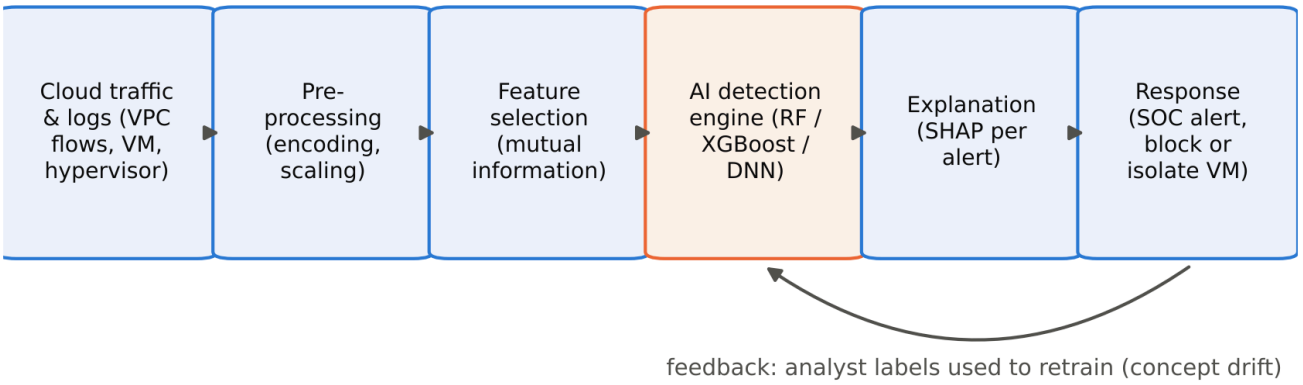


Fig. 1. Typical pipeline of an AI-based IDS in a cloud environment.

III. Review Methodology

The review was designed so that every cited claim can be checked by any reader at no cost. It followed four steps.

1. **Research questions.** RQ1: Which AI techniques are used for intrusion detection in cloud environments? RQ2: Which datasets and metrics are used to evaluate them? RQ3: How well do reported results hold under a strict evaluation protocol? RQ4: What gaps remain open?
2. **Sources and search terms.** A seed set of 24 citations from a preliminary survey was checked, and targeted searches were run on IEEE Xplore, arXiv, MDPI, SpringerOpen, Frontiers and CEUR Workshop Proceedings with the terms "intrusion detection" AND ("cloud" OR "network") AND ("machine learning" OR "deep learning" OR "artificial intelligence" OR "explainable" OR "federated").
3. **Inclusion criteria.** Full text freely accessible (open access, an arXiv preprint or an author-hosted copy); bibliographic record verifiable through a DOI, publisher page or indexing service; studies that propose, evaluate or survey AI-based network or cloud intrusion detection, or that introduce a benchmark dataset or method used by such studies; published 2009-2025.
4. **Exclusion criteria.** Records that could not be found in any index; papers without free full text; venues of low credibility; physical-security or video-surveillance "intrusion" studies; papers without enough technical detail to extract a model, dataset or result.

Fig. 2 summarises the selection. Of the 24 seed citations, 10 could not be verified in any index and 8 lacked free full text or came from low-credibility venues, leaving 6. A further 24 studies were added after full-text checking, giving 30 included studies. For every included experimental study, the model, dataset, evaluation protocol and headline result were extracted from the full text.

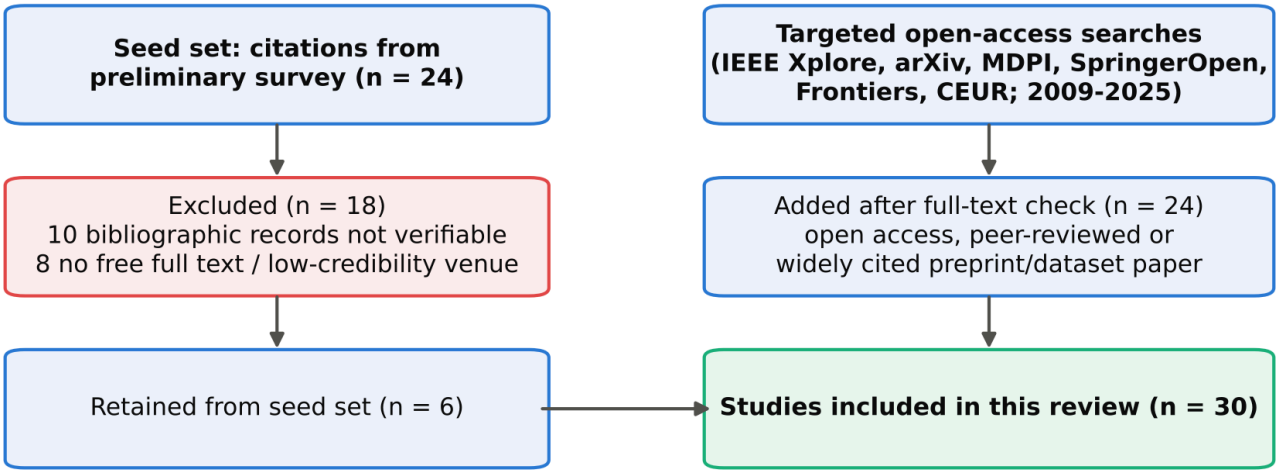


Fig. 2. Study selection flow.

IV. Taxonomy of AI Techniques for Cloud IDS

The reviewed studies fall into five overlapping families (Fig. 3). Surveys of the wider field reach a similar grouping: Pinto Neto et al. review 59 DL papers and 39 datasets across cloud, edge, IoT and industrial systems [6], Muneer et al. contrast ML, DL, federated learning and explainable AI [7], and Sharma et al. note that resampling and classifier ensembles are the most common ways to raise accuracy [8].

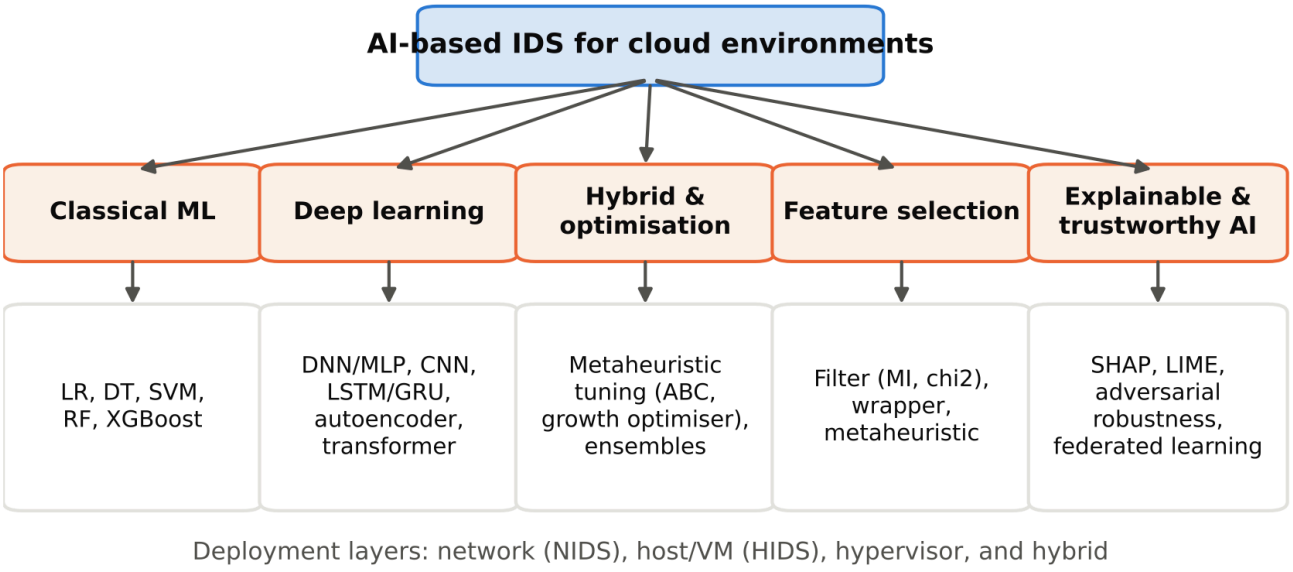


Fig. 3. Taxonomy of AI techniques for cloud intrusion detection.

A. Classical machine learning

Tree-based models dominate. Vinayakumar et al. compared a deep neural network with classical classifiers on several public benchmark datasets and proposed a scalable hybrid alerting framework; DNNs performed well against the classical baselines [9]. Alzahrani and Alenazi placed Decision Tree, Random Forest and XGBoost inside a software-defined networking controller and reached 95.95% multi-class accuracy on NSL-KDD using only 5 of its 41 features [10]. Attou et al. built a cloud IDS on Random Forest with feature engineering; on NSL-KDD they report 98.3% accuracy but a recall of 46.0%, which shows how accuracy alone can hide missed attacks [11]. Maodah et al. compared single classifiers with voting and stacking ensembles for cloud intrusion detection on NSL-KDD and report accuracy close to 100% for Random Forest, XGBoost and the ensembles [12]. XGBoost itself is a regularised gradient-boosting method designed to scale to large tabular data [13], which explains its popularity in this field.

B. Deep learning

DL models learn feature representations directly from traffic. Long et al. applied a Transformer encoder to CSE-CIC-IDS2018 and obtained 93.38-93.46% accuracy depending on the number of encoder layers [14]. Aljuaid and Alshamrani trained a one-dimensional CNN on CSE-CIC-IDS2018, a dataset generated on Amazon Web Services, using SMOTE to balance classes, and report 98.67% overall accuracy; the infiltration scenario remained the hardest at 92.43% [15]. DL generally needs more data and computation than tree ensembles, a trade-off examined in Section VII.

C. Hybrid and optimisation-based methods

Several cloud studies use metaheuristics to train or tune a model. Hajimirzaei and Navimipour trained a multilayer perceptron (MLP) with the artificial bee colony (ABC) algorithm, combined it with fuzzy clustering, and evaluated it in the CloudSim simulator on NSL-KDD, reporting lower error than prior methods [16]. Ali et al. extended this design and report 99.01% accuracy and a kappa statistic of 0.97 on NSL-KDD in CloudSim 5.0, alongside a neural classifier for malicious payloads drawn from Exploit-DB [17].

D. Feature selection

Feature selection reduces training and inference cost, which matters at cloud scale. Fatani et al. used a CNN for feature extraction and a modified Growth Optimizer for feature selection, and evaluated it on four public cloud and IoT intrusion datasets [18]. Rizvi et al. measured the cost side directly: on IoT-23, inside a virtual machine configured like a Raspberry Pi Zero, Decision Tree and Random Forest reached 99.99% accuracy, but XGBoost used 74.34% CPU and took 587.52 s to train against 8.26% and 1.332 s for a Decision Tree [19]. The same group surveys AI for network forensics more broadly [20].

E. Explainable and trustworthy AI

A high-scoring model that analysts cannot interpret is hard to deploy. SHAP attributes each prediction to the input features using Shapley values [21] and is now the most common explanation tool in IDS work. Khan et al. survey explainable IDS and their challenges [22], and Kumar and Thing used SHAP and TRUSTEE to inspect state-of-the-art IoT NIDS, finding that explanations expose what the models actually rely on [23]. Trustworthiness also covers privacy and robustness: federated learning trains a shared IDS without pooling raw tenant data [24], adversarial examples can be crafted to evade ML-based NIDS [25], and Transformer and large language model (LLM) based IDS are an emerging line of work [26].

V. Benchmark Datasets

Most reviewed studies still train and test on datasets recorded in a laboratory network, and most are not cloud traffic. Table I compares the datasets used by the reviewed studies; Ring et al. give a wider survey of dataset properties [27].

Table I. Benchmark datasets used in the reviewed studies.

Dataset	Year	Source of traffic	Size used	Features	Labels	Main limitation
NSL-KDD [28]	2009	Refinement of KDD Cup 99 (simulated 1998 military network)	125,973 train / 22,544 test (KDDTest+)	41	Normal + DoS, Probe, R2L, U2R	25+ years old; test file contains attack types absent from training
UNSW-NB15 [29]	2015	IXIA PerfectStorm testbed, UNSW Canberra	175,341 train / 82,332 test (official partition)	42	Normal + 9 attack categories	Synthetic traffic; some classes tiny (44 Worms test records)
CICIDS2017 [30]	2017	Canadian Institute for Cybersecurity testbed, 5 days	About 2.8 million flows	80+ flow features	Benign + 14 attack types	Heavy class imbalance
CSE-CIC-IDS2018 [14], [15]	2018	Generated on Amazon Web Services	12.2 million records in [14]	78 in [14]	Benign + 7 attack scenarios	Very large; few studies use the full file
IoT-23 [19]	2020	Captures from real IoT devices	Varies by study	Zeek flow fields	Benign + malware families	IoT rather than cloud traffic

Two properties of these datasets matter for the results in Section VII. First, NSL-KDD and UNSW-NB15 ship with fixed training and test files whose class mix differs: in the files used here, attacks make up 46.5% of NSL-KDD training records but 56.9% of its test records, and 68.1% of UNSW-NB15 training records but 55.1% of its test records. Second, only CSE-CIC-IDS2018 was generated on cloud infrastructure, so most "cloud IDS" results are measured on non-cloud traffic.

VI. Comparative Analysis

The reviewed experimental studies report accuracies between 93% and 100%, but they use different datasets, splits and metrics, so their numbers cannot be ranked against each other. Table II lists each study's model, data, evaluation protocol and headline result as stated in its full text.

Table II. Comparison of reviewed experimental studies (results as reported by the authors).

Ref.	Year	Model	Dataset	Evaluation protocol	Reported result	Cloud data?
[10]	2021	DT, RF, XGBoost on 5 features	NSL-KDD	Multi-class	95.95% accuracy	No (SDN)
[11]	2023	RF + feature engineering	NSL-KDD, Bot-IoT	Not stated as official split	98.3% accuracy, 46.0% recall (NSL-KDD)	No
[12]	2025	Voting and stacking ensembles	NSL-KDD	Not stated as official split	99.99% accuracy (RF, XGBoost)	No
[14]	2024	Transformer encoder	CSE-CIC-IDS2018	Random 7:3 split	93.38-93.46% accuracy	Yes (AWS)
[15]	2024	1D-CNN + SMOTE	CSE-CIC-IDS2018	Random 70:30 split	98.67% accuracy	Yes (AWS)
[16]	2019	MLP trained by ABC + fuzzy clustering	NSL-KDD in CloudSim	Simulation	Lower MAE and RMSE than baselines	Simulated
[17]	2023	MLP + ABC + fuzzy clustering	NSL-KDD in CloudSim 5.0	Simulation	99.01% accuracy, kappa 0.97	Simulated
[18]	2023	CNN + modified Growth Optimizer	4 public cloud and IoT datasets	Per dataset	Better than compared optimisers	Partly
[19]	2023	DT, RF, XGBoost, KNN, NB, CNN, LSTM, GRU	IoT-23	Constrained VM	99.99% accuracy (DT, RF)	No (edge)

Ref.	Year	Model	Dataset	Evaluation protocol	Reported result	Cloud data?
This work	2026	LR, DT, RF, XGBoost, MLP	NSL-KDD, UNSW-NB15	Official train/test files, 3 seeds	F1 0.798 (NSL-KDD), 0.897 (UNSW-NB15)	No

Three patterns stand out. First, only two studies [14], [15] use traffic generated on cloud infrastructure, and two more [16], [17] simulate a cloud around NSL-KDD. Second, accuracy is the headline metric almost everywhere, although [11] shows that 98.3% accuracy can coexist with 46.0% recall, meaning more than half of attacks were missed. Third, where the protocol is stated, studies evaluate on a random split of one data file rather than on a separately collected test set. Section VII tests how much that choice alone changes the result.

VII. Experimental Re-evaluation

On the official test files, the five models reach binary F1-scores of 0.742-0.798 on NSL-KDD and 0.849-0.897 on UNSW-NB15, far below the 99% common in the literature. This section describes the setup and the baseline results.

A. Setup

- **Data:** NSL-KDD (KDDTrain+ for training, KDDTest+ for testing) [28] and UNSW-NB15 (official training and testing partitions) [29], exactly as distributed. No record from a test file was used for training, tuning or scaling.
- **Pre-processing:** categorical fields one-hot encoded (NSL-KDD 41 to 122 features; UNSW-NB15 42 to 194 features), numeric fields standardised; both encoders fitted on training data only.
- **Models:** Logistic Regression (LR); Decision Tree (DT); Random Forest (RF, 100 trees); XGBoost (200 trees, depth 6, learning rate 0.1) [13]; and an MLP with hidden layers of 128 and 64 units and early stopping.
- **Protocol:** every binary result is the mean of three random seeds (0, 1, 2); standard deviations are shown where they are above zero (LR and XGBoost are deterministic here).
- **Metrics:** accuracy, precision, recall (detection rate), F1, false-positive rate (FPR), ROC-AUC, training time and inference time per flow.
- **Environment:** a 2 vCPU, 8 GB cloud container; scikit-learn 1.9.1, XGBoost 3.4.1, SHAP 0.54.1. Code and outputs are released with the paper.

B. Binary detection on the official test sets

Table III. Binary detection on official test sets (mean of 3 seeds).

Dataset	Model	Accuracy	Precision	Recall	F1	FPR	ROC-AUC	Train (s)	Inference (µs/flow)
NSL-KDD	LR	0.754	0.917	0.623	0.742	0.074	0.792	4.5	0.66
NSL-KDD	DT	0.803 ± 0.007	0.963	0.681	**0.798 ± 0.010**	0.034	0.823	2.0	0.54
NSL-KDD	RF	0.774 ± 0.006	0.969	0.624	0.759 ± 0.008	**0.027**	0.960	5.4	5.15
NSL-KDD	XGBoost	0.794	0.968	0.659	0.784	0.028	**0.965**	3.8	1.90
NSL-KDD	MLP	0.784 ± 0.012	0.942	0.661	0.777 ± 0.012	0.055	0.900	30.6	1.20
UNSW-NB15	LR	0.809	0.753	0.972	0.849	0.391	0.956	10.3	0.43
UNSW-NB15	DT	0.863	0.824	0.955	0.885	**0.250**	0.855	3.6	0.23
UNSW-NB15	RF	0.871	0.818	0.985	0.894	0.268	0.978	11.8	6.77
UNSW-NB15	XGBoost	0.875	0.824	0.985	**0.897**	0.258	**0.984**	7.1	1.72
UNSW-NB15	MLP	0.859 ± 0.004	0.807	0.978	0.884 ± 0.002	0.287	0.977	58.8	2.32

The two datasets fail in opposite directions. On NSL-KDD, precision is high (0.92-0.97) but recall is low (0.62-0.68): about one attack in three goes undetected. On UNSW-NB15, recall is high (0.95-0.99) but the FPR is 0.25-0.39, so one in four normal flows raises a false alarm. No single model dominates: DT gives the best NSL-KDD F1, XGBoost the best UNSW-NB15 F1 and ROC-AUC, and the MLP costs 5-16 times more training time than the tree models without beating them.

C. The random-split pitfall

To test whether the evaluation protocol explains the gap with the literature, DT, RF and XGBoost were also trained on 80% of each training file and tested on the remaining 20% (stratified random split). Fig. 4 compares the two protocols. On NSL-KDD, the random split yields 0.998-0.999 accuracy against 0.778-0.801 on the official test file, a drop of about 20 percentage points. On UNSW-NB15 the drop is 8-9 points (0.950-0.959 against 0.863-0.875). A random split can put near-duplicate flows from the same capture on both sides of the split, so the model is tested on traffic it has effectively already seen; the

official test files contain different traffic and, for NSL-KDD, attack types absent from training. Results reported on a random split of one file therefore measure memorisation as much as detection, a pitfall also described in [4], [5].

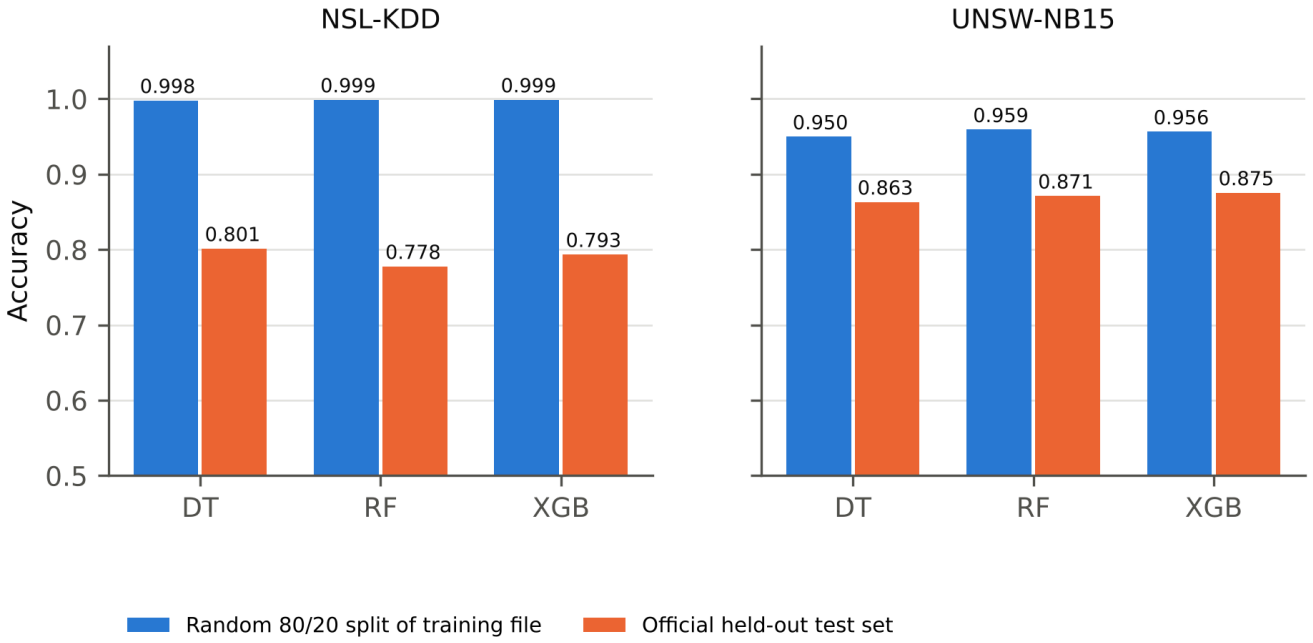


Fig. 4. Accuracy under a random 80/20 split of the training file versus the official held-out test set.

D. Multi-class detection

Table IV. Multi-class detection on official test sets (seed 0).

Dataset	Model	Accuracy	Macro-F1	Weighted-F1
NSL-KDD (5 classes)	LR	0.761	0.527	0.716
NSL-KDD (5 classes)	DT	0.764	0.528	0.726
NSL-KDD (5 classes)	RF	0.750	0.495	0.709
NSL-KDD (5 classes)	XGBoost	**0.779**	0.541	**0.743**
NSL-KDD (5 classes)	MLP	0.772	**0.564**	0.739
UNSW-NB15 (10 classes)	LR	0.698	0.340	0.706
UNSW-NB15 (10 classes)	DT	0.733	0.498	0.762
UNSW-NB15 (10 classes)	RF	0.754	0.463	0.775
UNSW-NB15 (10 classes)	XGBoost	**0.770**	**0.507**	**0.783**
UNSW-NB15 (10 classes)	MLP	0.759	0.438	0.763

Macro-F1, which weights every class equally, stays at or below 0.564 because rare classes are almost never recognised. On NSL-KDD the best recall for U2R is 0.105 and for R2L 0.112 (both MLP). On UNSW-NB15 the best recall is 0.081 for Analysis (XGBoost), 0.146 for Backdoor and 0.174 for DoS (both DT). These are the low-volume, high-impact attacks an operator most needs to catch.

E. Feature selection and cost

Features were ranked by mutual information (MI) on the training data, and RF and XGBoost were retrained on the top 10, 20 and 30 (Fig. 5). Keeping 30 features cut XGBoost training time from 3.76 s to 1.53 s on NSL-KDD (-59%) and from 7.71 s to 2.03 s on UNSW-NB15 (-74%), while F1 moved from 0.781 to 0.789 and from 0.897 to 0.893. Ten features cost 0.04 F1 on NSL-KDD but under 0.01 on UNSW-NB15. Random Forest gained little time from selection, most likely because its cost depends more on the number of training rows than on the number of features. The top MI features were byte counts and connection-rate features (NSL-KDD: src_bytes, dst_bytes, diff_srv_rate, same_srv_rate; UNSW-NB15: sbytes, sttl, dbytes, dttl).

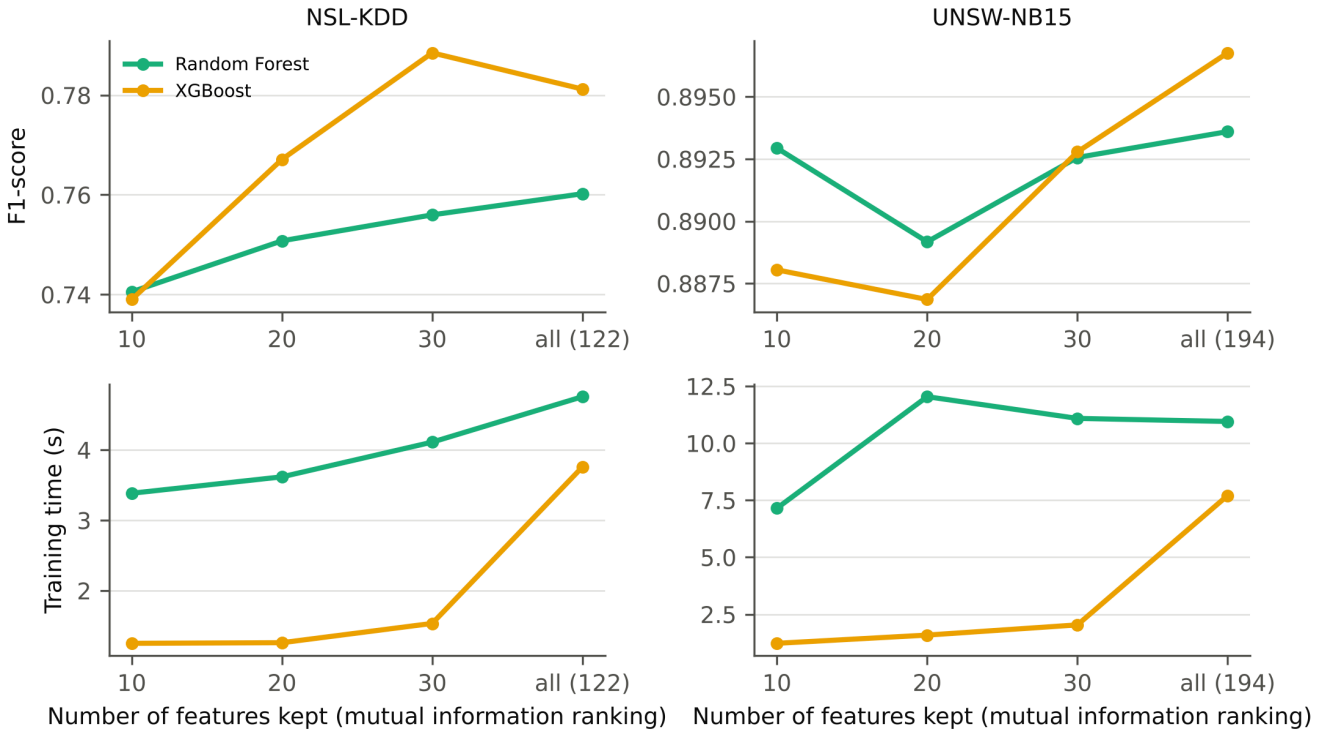


Fig. 5. F1-score and training time against the number of features kept.

F. Explanations

SHAP values [21] were computed for XGBoost on 2,000 test flows per dataset (Fig. 6). On NSL-KDD, the source byte count dominates (mean |SHAP| 3.43), followed by connection-count features such as dst_host_srv_count (1.35) and count (0.92), which matches how DoS and probing attacks behave. On UNSW-NB15 a single feature, the source time-to-live sttl, dominates (3.21, more than four times the next feature). A TTL value describes the testbed hosts that generated the traffic more than the attack itself, so this dependence is a warning that the model may have learned a property of the dataset rather than of attacks; Section VIII tests how easily such dependence can be exploited.

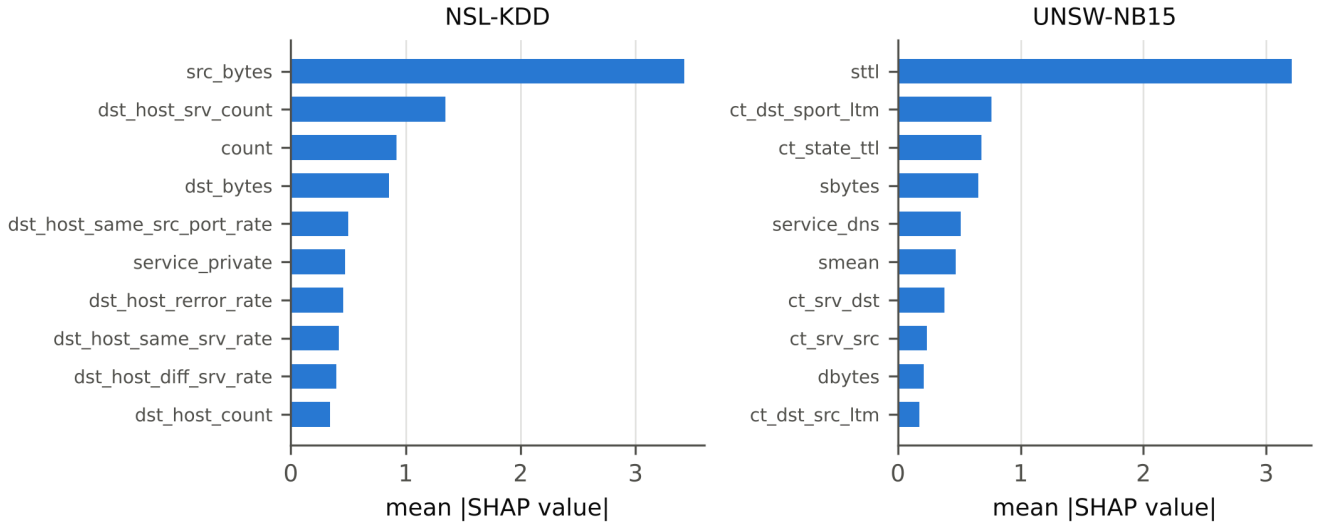


Fig. 6. Top-10 features by mean absolute SHAP value for XGBoost.

VIII. RELIC-IDS: Reliability Evaluation of Cloud IDS

Accuracy on a test file says little about how an IDS behaves once deployed. We therefore propose **RELIC-IDS** (Reliability Evaluation for Learning-based Intrusion detection in Cloud), a set of five tests that each answer an operational question, plus a reporting checklist. All five were run on the models of Section VII; the main result is that every model fails at least one test badly even when its F1 looks acceptable.

A. R1 - Zero-day detection: does the model catch attack types it was never trained on?

The NSL-KDD test file contains 17 attack types that never appear in the training file (3,750 records) alongside 9,083 records of known types. Fig. 7 shows that every model detects known attack types at 0.71-0.79 but unseen types at only 0.28-0.42, roughly half the rate. Random Forest, the most cited model in the reviewed literature, is the weakest at 0.28. Per type, XGBoost detected 82% of apache2 and 99.7% of saint, but none of the 331 snmpguess, 293 mailbomb or 178 snmpgetattack records. The claim that AI-based IDS "detect zero-day attacks" holds only partially, and depends on how similar the new attack is to known ones.

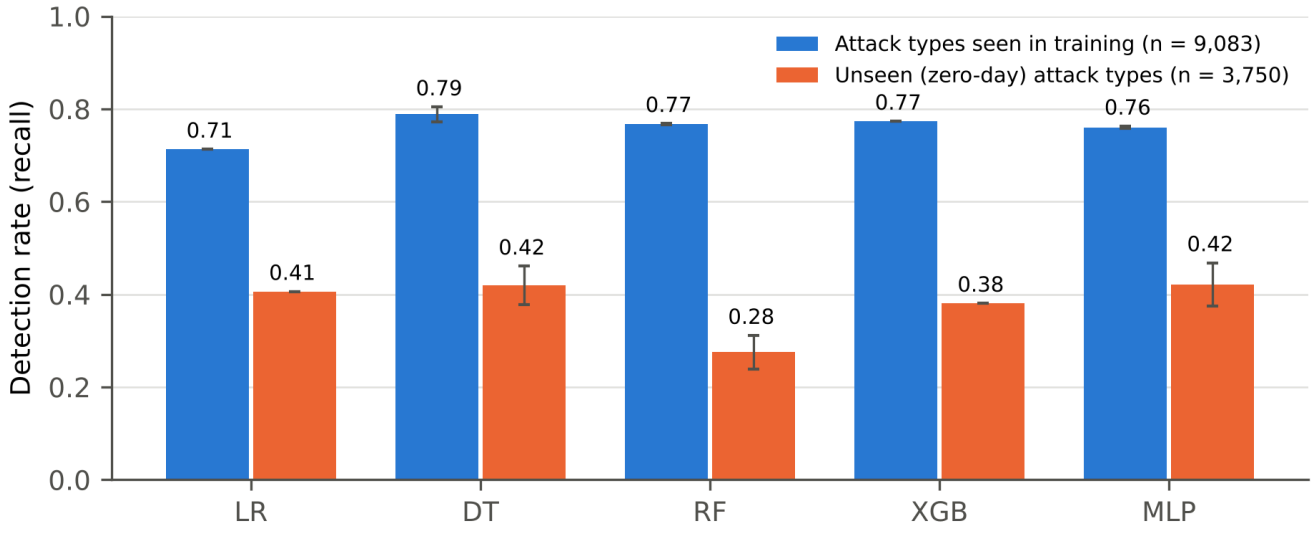


Fig. 7. Detection of attack types seen in training versus unseen (zero-day) types on NSL-KDD (mean and standard deviation of 3 seeds).

B. R2 - False-alarm control: can an operator hold the false-positive rate at a chosen level?

A SOC usually fixes an acceptable false-alarm rate and accepts the recall that follows. We chose the decision threshold on a 20% validation split of the training file so that 5% or 1% of normal validation flows would be flagged, then applied it to the official test file (Table V).

Table V. Decision thresholds set on validation data and their effect on the official test set.

Dataset	Model	FPR target (validation)	Threshold	Test FPR	Test recall	Test F1
NSL-KDD	RF	default	0.500	0.027	0.603	0.743
NSL-KDD	RF	5%	0.010	0.151	0.984	0.938
NSL-KDD	RF	1%	0.080	0.100	0.881	0.901
NSL-KDD	XGBoost	default	0.500	0.028	0.657	0.783
NSL-KDD	XGBoost	5%	0.002	0.109	0.890	0.903
NSL-KDD	XGBoost	1%	0.014	0.038	0.803	0.877
UNSW-NB15	RF	default	0.500	0.276	0.985	0.891
UNSW-NB15	RF	5%	0.620	0.193	0.966	0.910
UNSW-NB15	RF	1%	0.851	0.052	0.892	0.923
UNSW-NB15	XGBoost	default	0.500	0.259	0.986	0.897
UNSW-NB15	XGBoost	5%	0.630	0.178	0.969	0.917
UNSW-NB15	XGBoost	1%	0.867	0.045	0.910	0.935

Two findings follow. First, the threshold matters as much as the model: moving it from the default 0.5 raised test F1 by up to 0.19 (RF on NSL-KDD, 0.743 to 0.938), and on UNSW-NB15 a 1% validation target cut the test FPR of XGBoost from 0.259 to 0.045 while F1 rose from 0.897 to 0.935. Second, targets set on in-distribution validation data did not hold on the test file: the realised FPR was 2.2 to 10 times the target. A deployed cloud IDS therefore needs thresholds recalibrated on recent production traffic, not fixed at training time.

C. R3 - Rare attacks: does class balancing rescue the attacks that matter most?

We retrained the multi-class XGBoost with no balancing, with balanced sample weights and with SMOTE oversampling of the training data only (Table VI; Random Forest results were similar or lower and are in the released outputs).

Table VI. Effect of class balancing on rare-class recall (multi-class XGBoost, official test sets).

Dataset	Balancing	Macro-F1	Recall of rare classes	Normal recall
NSL-KDD	None	0.541	R2L 0.094, U2R 0.060	0.971
NSL-KDD	Balanced weights	0.549	R2L 0.114, U2R 0.060	0.972
NSL-KDD	SMOTE	**0.581**	R2L 0.132, U2R 0.120	0.972
UNSW-NB15	None	**0.507**	Analysis 0.081, Backdoor 0.065, DoS 0.106, Worms 0.455	0.763
UNSW-NB15	Balanced weights	**0.507**	Analysis 0.066, Backdoor 0.828, DoS 0.185, Worms 0.864	0.598
UNSW-NB15	SMOTE	0.489	Analysis 0.037, Backdoor 0.237, DoS 0.508, Worms 0.773	0.682

Balancing helps specific classes but is no general fix. On NSL-KDD, SMOTE doubled U2R recall to 0.120, yet nine in ten U2R and R2L attacks are still missed. On UNSW-NB15, balanced weights raised Backdoor recall from 0.065 to 0.828 and Worms from 0.455 to 0.864, but normal recall fell from 0.763 to 0.598, so false alarms rose sharply and macro-F1 did not change. Rare-class gains are bought with false alarms, which is exactly the trade-off R2 asks operators to make explicit.

D. R4 - Adversarial mimicry: how easily can an attacker evade the model?

We took the attacks that XGBoost correctly detected (8,453 on NSL-KDD and 44,634 on UNSW-NB15) and moved their k most influential numeric features by SHAP (k = 1, 2, 3) toward the median of benign training traffic, using $x' = (1 - \alpha)x + \alpha m$ with strength α from 0 to 1 (Fig. 8). On UNSW-NB15, changing only the IP time-to-live (sttl) to the benign median of 31 (attacks have a median of 254) let 99.8% of detected attacks evade the model; only 0.19% were still flagged. Setting a packet's TTL is trivial for an attacker. On NSL-KDD, changing src_bytes alone left 59.7% of attacks detected, and changing the top three features left 11.4%. The features SHAP ranks highest are thus also the model's weakest points, and an IDS that relies on one dominant feature can be bypassed with a single change.

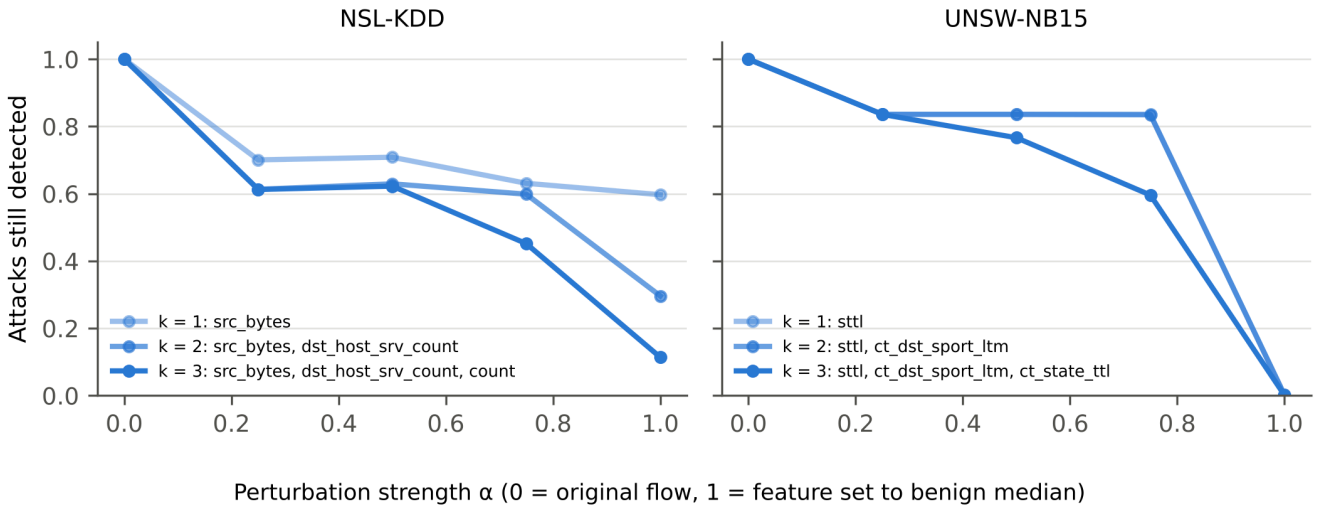


Fig. 8. Share of detected attacks that remain detected after their top-k SHAP features are moved toward the benign median (XGBoost). On UNSW-NB15 the k = 1 and k = 2 curves coincide.

E. R5 - Deployment cost: what does each model cost to run?

Table VII reports model size, latency for one flow and batch throughput on a single CPU thread (UNSW-NB15, 194 features).

Table VII. Deployment cost per model (single thread, UNSW-NB15).

Model	Model size	Latency per flow (ms)	Batch throughput (flows/s)
LR	1.4 KB	0.23	6,235,621
DT	890 KB	0.11	4,649,081
RF	108 MB	6.23	121,826
XGBoost	517 KB	0.32	310,120
MLP	399 KB	0.14	209,854

XGBoost delivers the best F1 at 517 KB and about 310,000 flows per second on one thread, while Random Forest is 214 times larger, about 19 times slower per flow and 2.5 times slower in batch for a lower F1. For high-rate cloud links or resource-constrained edge nodes, model size and latency should be reported alongside accuracy, as [19] also argues.

F. The RELIC-IDS reporting checklist

We recommend that every AI-based cloud IDS paper report the following:

1. Results on an official, time-separated or capture-separated test set, never only a random split of one file.

2. The mean and spread over at least three random seeds.
3. Recall, precision, F1, FPR and ROC-AUC, not accuracy alone.
4. Per-class recall for multi-class tasks, with rare classes named.
5. Detection of attack types absent from training (R1), where the data allows it.
6. The decision threshold, how it was chosen, and the FPR at that operating point (R2).
7. Any balancing method and its effect on normal-class recall (R3).
8. Feature attributions (e.g. SHAP) and a robustness check on the top features (R4).
9. Training time, model size, latency and throughput on stated hardware (R5).
10. Code, versions and data sources sufficient to reproduce every number.

IX. Research Gaps

The review and the experiments point to the same open problems. Each gap below is tied to the evidence that supports it.

#	Gap	Evidence in this paper
G1	No shared evaluation protocol; random splits inflate results	Accuracy falls by about 20 points on NSL-KDD and 8-9 points on UNSW-NB15 when the official test file replaces a random split (Section VII-C)
G2	Little cloud-generated training data	Only 2 of 9 experimental studies in Table II use traffic generated on cloud infrastructure [14], [15]
G3	Weak detection of unseen attacks	Detection of attack types absent from training is far below that of known types (Section VIII-A)
G4	Rare, high-impact attacks are missed	Best recall 0.105 for U2R and 0.081 for Analysis; macro-F1 at most 0.58 even with SMOTE (Sections VII-D, VIII-C)
G5	False-alarm rates are not controlled	FPR of 0.25-0.39 on UNSW-NB15; FPR targets set on validation data are missed on the test set (Section VIII-B)
G6	Explanations are produced but rarely acted on	SHAP shows reliance on sttl, a testbed property (Section VII-F)
G7	Adversarial robustness is rarely tested	Mimicry of a few influential features lets detected attacks evade the model (Section VIII-D) [25]

#	Gap	Evidence in this paper
G8	Computational cost is seldom reported	Only [19] reports CPU use; costs differ widely across models (Section VIII-E)
G9	Tenant privacy limits data sharing	Federated learning is proposed but rarely evaluated on cloud data [24]

X. Future Directions

1. **Cloud-native, time-ordered benchmarks.** Datasets captured from real virtual networks, flow logs and hypervisors, released with fixed time-ordered train and test periods so that concept drift is part of the test.
2. **A reporting standard.** Results on an official or time-separated test set, per-class recall, FPR at a stated operating point, cost per flow, and the seed count, as in the RELIC-IDS checklist (Section VIII-F).
3. **Open-set and anomaly-aware detection.** Combining a classifier for known attacks with an anomaly detector (for example an autoencoder) for traffic unlike anything seen in training, aimed at G3.
4. **Cost-sensitive learning and online recalibration.** Treating a missed attack and a false alarm as errors of different cost, and re-setting thresholds from analyst feedback as traffic drifts (Fig. 1).
5. **Federated and privacy-preserving IDS.** Training across tenants or providers without pooling raw traffic [24].
6. **Robust training under realistic constraints.** Adversarial training and feature constraints that reflect what an attacker can actually change in a flow [25].
7. **Explainability in the analyst workflow.** Using SHAP-style explanations to triage alerts and to detect models that rely on dataset artefacts [21]-[23].
8. **Transformer and LLM-assisted detection.** Sequence models over flows [14] and LLMs for alert summarisation and triage [26], evaluated under the same strict protocol.
9. **Lightweight models for the edge-cloud continuum.** Small tree models and selected feature sets for gateways and constrained nodes [19].

XI. Limitations of This Study

- **Datasets.** NSL-KDD and UNSW-NB15 are laboratory datasets, not cloud traffic. CSE-CIC-IDS2018, which was generated on AWS, could not be downloaded from the experiment environment, so a cloud-native evaluation remains future work.

- **Models.** Sequence models such as CNN, LSTM and Transformer were not trained; the MLP is the only neural model. Hyperparameters were fixed, not tuned, so each model's absolute score could rise with tuning, although tuning cannot remove the protocol gap in G1.
- **Adversarial test.** The mimicry attack changes feature values directly. A real attacker must change packets, which may make some changes impossible and others easier.
- **Timing.** Times were measured on a shared 2 vCPU container and should be read as relative, not absolute.
- **Review scope.** Only 30 freely accessible studies were included, screened by a single reviewer, so relevant subscription-only work may be missing.

XII. Conclusion

AI-based intrusion detection for cloud environments is widely reported to exceed 99% accuracy, but this study shows that such figures depend heavily on how a model is evaluated. Reviewing 30 open-access studies and re-evaluating five models on the official NSL-KDD and UNSW-NB15 test files, we found best binary F1-scores of 0.798 and 0.897, and showed that a random split of the training file alone raises accuracy to 0.998 and 0.959. The proposed RELIC-IDS tests exposed further weaknesses that accuracy hides: unseen attack types were detected at about half the rate of known ones, false-alarm targets set at training time were missed by a factor of 2.2 to 10, class balancing traded rare-class recall for false alarms, a single TTL change let 99.8% of detected UNSW-NB15 attacks evade XGBoost, and model size varied by a factor of more than 70,000. XGBoost offered the best balance of detection and cost in our tests and gained most from feature selection and threshold recalibration, but no model was robust. Progress now depends less on new architectures than on cloud-native time-ordered datasets, strict evaluation, threshold recalibration in production and robustness testing, and the RELIC-IDS checklist is offered as a practical starting point.

Code and Data Availability

All code, result files and figures are available at https://github.com/jayveerv/relic-ids. NSL-KDD and UNSW-NB15 are publicly available from their original sources [28], [29]. Every number in Tables III-VII and Figs. 4-8 is produced by the two released scripts.

References

[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, art. 20, 2019, doi: 10.1186/s42400-019-0038-7.

[2] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Trans. Emerging Telecommun. Technol.*, vol. 32, no. 1, art. e4150, 2021, doi: 10.1002/ett.4150.

[3] S. S. Nasim, P. Pranav, and S. Dutta, "A systematic literature review on intrusion detection techniques in cloud computing," *Discover Computing*, vol. 28, art. 107, 2025, doi: 10.1007/s10791-025-09641-y.

[4] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305-316, doi: 10.1109/SP.2010.25.

[5] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022. arXiv:2010.09470.

[6] E. C. Pinto Neto, S. Iqbal, S. Buffett, M. Sultana, and A. Taylor, "Deep learning for intrusion detection in emerging technologies: A comprehensive survey and new perspectives," *Artif. Intell. Rev.*, vol. 58, no. 10, art. 340, 2025, doi: 10.1007/s10462-025-11346-z.

[7] S. Muneer, U. Farooq, A. Athar, M. A. Raza, T. M. Ghazal, and S. Sakib, "A critical review of artificial intelligence based approaches in intrusion detection: A comprehensive analysis," *J. Eng.*, vol. 2024, art. 3909173, 2024, doi: 10.1155/2024/3909173.

[8] V. Sharma, D. Shah, S. Sharma, and S. Gautam, "Artificial intelligence based intrusion detection system - A detailed survey," *ITM Web Conf.*, vol. 65, art. 04002, 2024, doi: 10.1051/itmconf/20246504002.

[9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525-41550, 2019, doi: 10.1109/ACCESS.2019.2895334.

[10] A. O. Alzahrani and M. J. F. Alenazi, "Designing a network intrusion detection system based on machine learning for software defined networks," *Future Internet*, vol. 13, no. 5, art. 111, 2021, doi: 10.3390/fi13050111.

[11] H. Attou, A. Guezzaz, S. Benkirane, M. Azrour, and Y. Farhaoui, "Cloud-based intrusion detection approach using machine learning techniques," *Big Data Mining and Analytics*, vol. 6, no. 3, pp. 311-320, 2023, doi: 10.26599/BDMA.2022.9020038.

[12] K. A. Maodah, S. Alhomdy, and F. Thabit, "Detecting intrusions in cloud-based ensembles: Evaluating voting and stacking methods with machine learning classifiers," *Front. Comput. Sci.*, vol. 7, art. 1623375, 2025, doi: 10.3389/fcomp.2025.1623375.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785-794. arXiv:1603.02754.

[14] Z. Long, H. Yan, G. Shen, X. Zhang, H. He, and L. Cheng, "A Transformer-based network intrusion detection approach for cloud security," *J. Cloud Comput.*, vol. 13, art. 5, 2024, doi: 10.1186/s13677-023-00574-9.

[15] W. H. Aljuaid and S. S. Alshamrani, "A deep learning approach for intrusion detection systems in cloud computing environments," *Appl. Sci.*, vol. 14, no. 13, art. 5381, 2024, doi: 10.3390/app14135381.

[16] B. Hajimirzaei and N. J. Navimipour, "Intrusion detection for cloud computing using neural networks and artificial bee colony optimization algorithm," *ICT Express*, vol. 5, no. 1, pp. 56-59, 2019, doi: 10.1016/j.icte.2018.01.014.

[17] M. Ali, A. Razaque, D. M. Yeskendirova, T. A. Nurlybayev, N. Y. Askarbekova, and Z. A. Kashaganova, "Enhancing cloud computing security through AI-based intelligent intrusion detection leveraging neural networks and artificial bee colony optimization," in *Proc. 8th Int. Conf. Digital Technologies in Education, Science and Industry (DTESI)*, CEUR Workshop Proc., vol. 3680, 2023. Online.

[18] A. Fatani et al., "Enhancing intrusion detection systems for IoT and cloud environments using a growth optimizer algorithm and conventional neural networks," *Sensors*, vol. 23, no. 9, art. 4430, 2023, doi: 10.3390/s23094430.

[19] S. Rizvi, M. Scanlon, J. McGibney, and J. Sheppard, "An evaluation of AI-based network intrusion detection in resource-constrained environments," in *Proc. IEEE 14th Annu. Ubiquitous Computing, Electronics and Mobile Communication Conf. (UEMCON)*, 2023. Online.

[20] S. Rizvi, M. Scanlon, J. McGibney, and J. Sheppard, "Application of artificial intelligence to network forensics: Survey, challenges and future directions," *IEEE Access*, vol. 10, pp. 110362-110384, 2022, doi: 10.1109/ACCESS.2022.3214506.

[21] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. arXiv:1705.07874.

[22] N. Khan, K. Ahmad, A. Al Tamimi, M. M. Alani, A. Bermak, and I. Khalil, "Explainable AI-based intrusion detection system for Industry 5.0: An overview of the literature, associated challenges, the existing solutions, and potential research directions," 2024. arXiv:2408.03335.

[23] A. Kumar and V. L. L. Thing, "Evaluating the explainability of state-of-the-art machine learning-based IoT network intrusion detection systems," 2024. arXiv:2408.14040.

[24] S. Agrawal et al., "Federated learning for intrusion detection system: Concepts, challenges and future directions," *Comput. Commun.*, vol. 195, pp. 346-361, 2022. arXiv:2106.09527.

[25] E. Alhajjar, P. Maxwell, and N. Bastian, "Adversarial machine learning in network intrusion detection systems," *Expert Syst. Appl.*, vol. 186, art. 115782, 2021. arXiv:2004.11898.

[26] H. Kheddar, "Transformers and large language models for efficient intrusion detection systems: A comprehensive survey," 2024. arXiv:2408.07583.

[27] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Comput. Secur.*, vol. 86, pp. 147-167, 2019. arXiv:1903.02460.

[28] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1-6, doi: 10.1109/CISDA.2009.5356528.

[29] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Military Communications and Information Systems Conf. (MilCIS)*, 2015, pp. 1-6, doi: 10.1109/MilCIS.2015.7348942.

[30] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108-116, doi: 10.5220/0006639801080116.